

CUSTOMIZE BEHAVIOR

Slash Commands in the SDK

Learn how to use slash commands to control Claude Code sessions through the SDK

Slash commands provide a way to control Claude Code sessions with special commands that start with `/`. These commands can be sent through the SDK to perform actions like compacting context, listing context usage, or invoking custom commands. Only commands that work without an interactive terminal are dispatchable through the SDK; the `system/init` message lists the ones available in your session.

Discovering Available Slash Commands

The Claude Agent SDK provides information about available slash commands in the system initialization message. Access this information when your session starts:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Hello Claude",
  options: { maxTurns: 1 }
})) {
  if (message.type === "system" && message.subtype === "init") {
    console.log("Available slash commands:", message.slash_commands);
    // Includes built-in commands plus bundled skills, for example:
    // ["clear", "compact", "context", "usage", "code-review", "verify", ...]
  }
}
```

Sending Slash Commands

Send slash commands by including them in your prompt string, just like regular text. Commands that act on conversation history, such as `/compact`, need prior messages to work with, so the examples below ask a question first and then send the command as a follow-up to the same conversation:

```

import { query } from "@anthropic-ai/claude-agent-sdk";

// Build up conversation history first
try {
  for await (const message of query({
    prompt: "What does the README in this directory cover?",
    options: { maxTurns: 2 }
  })) {
    if (message.type === "result" && message.subtype === "success") {
      console.log(message.result);
    }
  }
} catch (error) {
  // A single-shot query() throws after yielding an error result,
  // so the follow-up query below still runs.
  console.error(`Session ended with an error: ${error}`);
}

// Send a slash command as a follow-up to the same conversation
for await (const message of query({
  prompt: "/compact",
  options: { continue: true, maxTurns: 1 }
})) {
  if (message.type === "result") {
    console.log("Command executed, result subtype:", message.subtype);
    // Example output: Command executed, result subtype: success
  }
}

```

- ❗ A query can end with an error result, for example when the `maxTurns` / `max_turns` limit is reached before the work completes. The final result message then has `is_error: true` and an error subtype such as `error_max_turns` instead of `success`.

After yielding that final result message, the SDK raises an error, because the CLI process exits with a non-zero code.

Wrap the loop in a `try / catch` in TypeScript or `try / except` in Python if your command might hit the limit, as shown in [Single Message Input](#), or set `maxTurns` high enough for the work to complete. In Python, catch `Exception`: the SDK surfaces error results as a plain `Exception`.

Common Slash Commands

`/compact` - Compact conversation history

The `/compact` command reduces the size of your conversation history by summarizing older messages while preserving important context. Compaction needs an existing conversation with at least two prior exchanges to summarize. This example has a conversation first, then compacts it and reads the `compact_boundary` system message that reports the result:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Compaction needs existing history, so have a conversation first
try {
  for await (const message of query({
    prompt: "Explain what this project does",
    options: { maxTurns: 2 }
  })) {
    if (message.type === "result" && message.subtype === "success") {
      console.log(message.result);
    }
  }
} catch (error) {
  // A single-shot query() throws after yielding an error result,
  // so the follow-up query below still runs.
  console.error(`Session ended with an error: ${error}`);
}

// Compact the same conversation
for await (const message of query({
  prompt: "/compact",
  options: { continue: true, maxTurns: 1 }
})) {
  if (message.type === "system" && message.subtype === "compact_boundary") {
    console.log("Compaction completed");
    console.log("Pre-compaction tokens:",
message.compact_metadata.pre_tokens);
    console.log("Trigger:", message.compact_metadata.trigger);
    // Example output:
    // Compaction completed
    // Pre-compaction tokens: 1842
    // Trigger: manual
  }
}
```

❗ A `compact_boundary` message only arrives when compaction ran. With nothing to summarize, `/compact` reports the reason instead of raising: the run still ends with a `success` result, no `compact_boundary` message is emitted, and the result text carries the message, for example `Not enough messages to compact.` after a single short exchange. A fresh one-shot `query()` call starts with empty context, so use this pattern in a session with prior turns, for example in [streaming input mode](#) or when resuming a session.

`/clear` - Reset conversation context

The `/clear` command resets the conversation to an empty context, so subsequent prompts start with no prior conversation history. The previous conversation remains on disk and can be returned to by passing its session ID to the [resume option](#).

This is useful in [streaming input mode](#), where you send multiple prompts over a single connection. For one-shot `query()` calls, each call already starts with empty context, so sending `/clear` has no practical effect; start a new `query()` instead.

❗ `/clear` in the SDK requires Claude Code v2.1.117 or later. In earlier versions it is omitted from `slash_commands`.

Creating Custom Slash Commands

In addition to using built-in slash commands, you can create your own custom commands that are available through the SDK. Custom commands are defined as markdown files in specific directories, similar to how subagents are configured.

❗ The `.claude/commands/` directory is the legacy format. The recommended format is `.claude/skills/<name>/SKILL.md`, which supports the same slash-command invocation (`/name`) plus autonomous invocation by Claude. See [Skills](#) for the current format. The CLI continues to support both formats, and the examples below remain accurate for `.claude/commands/`.

File Locations

Custom slash commands are stored in designated directories based on their scope:

- **Project commands:** `.claude/commands/` - Available only in the current project (legacy; prefer `.claude/skills/`)
- **Personal commands:** `~/.claude/commands/` - Available across all your projects (legacy; prefer

```
~/.claude/skills/ )
```

File Format

Each custom command is a markdown file where:

- The filename (without `.md` extension) becomes the command name
- The file content defines what the command does
- Optional YAML frontmatter provides configuration

Basic Example

Create the `.claude/commands` directory in your project if it doesn't exist, then create `.claude/commands/refactor.md` :

```
Refactor the selected code to improve readability and
maintainability.
Focus on clean code principles and best practices.
```

This creates the `/refactor` command that you can use through the SDK.

With Frontmatter

Create `.claude/commands/security-check.md` :

```
---
allowed-tools: Read, Grep, Glob
description: Run security vulnerability scan
model: claude-opus-4-8
---
```

Analyze the codebase for security vulnerabilities including:

- SQL injection risks
- XSS vulnerabilities
- Exposed credentials
- Insecure configurations

Using Custom Commands in the SDK

Once defined in the filesystem, custom commands are automatically available through the SDK:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Use a custom command
try {
  for await (const message of query({
    prompt: "/refactor src/auth/login.ts",
    options: { maxTurns: 3 }
  })) {
    if (message.type === "assistant") {
      console.log("Refactoring suggestions:", message.message);
    }
  }
} catch (error) {
  // A single-shot query() throws after yielding an error result,
  // so the second query below still runs.
  console.error(`Session ended with an error: ${error}`);
}

// Custom commands appear in the slash_commands list
for await (const message of query({
  prompt: "Hello",
  options: { maxTurns: 1 }
})) {
  if (message.type === "system" && message.subtype === "init") {
    console.log("Available commands:", message.slash_commands);
    // Includes built-in commands plus bundled skills and your custom
    // commands, for example:
    // ["clear", "compact", "context", "usage", "code-review", "verify",
    "refactor", "security-check", ...]
  }
}
```

Advanced Features

Arguments and Placeholders

Custom commands support dynamic arguments using placeholders:

Create `.claude/commands/fix-issue.md` :

```
---  
argument-hint: [issue-number] [priority]  
description: Fix a GitHub issue  
---
```

Fix issue # $\$0$ with priority $\$1$.
Check the issue description and implement the necessary changes.

Use in SDK:

```
import { query } from "@anthropic-ai/claude-agent-sdk";  
  
// Pass arguments to custom command  
for await (const message of query({  
  prompt: "/fix-issue 123 high",  
  options: { maxTurns: 5 }  
})) {  
  // Command will process with $0="123" and $1="high"  
  if (message.type === "result" && message.subtype === "success") {  
    console.log("Issue fixed:", message.result);  
  }  
}
```

Bash Command Execution

Custom commands can execute bash commands and include their output:

Create `.claude/commands/git-commit.md` :

```
---
allowed-tools: Bash(git add *), Bash(git status *), Bash(git
commit *)
description: Create a git commit
---
```

Context

- Current status: `!`git status``
- Current diff: `!`git diff HEAD``

Task

Create a git commit with appropriate message based on the changes.

File References

Include file contents using the `@` prefix:

Create `.claude/commands/review-config.md` :

```
---
description: Review configuration files
---
```

Review the following configuration files for issues:

- Package config: `@package.json`
- TypeScript config: `@tsconfig.json`
- Environment config: `@.env`

Check for security issues, outdated dependencies, and misconfigurations.

Organization with Namespacing

Organize commands in subdirectories for better structure:


```
.claude/commands/
├── frontend/
│   ├── component.md      # Creates /component
│   │   (project:frontend)
│   └── style-check.md    # Creates /style-check
│       (project:frontend)
├── backend/
│   ├── api-test.md       # Creates /api-test
│   │   (project:backend)
│   └── db-migrate.md     # Creates /db-migrate
│       (project:backend)
└── review.md             # Creates /review (project)
```

The subdirectory appears in the command description but doesn't affect the command name itself.

Practical Examples

Pull Request Review Command

Create `.claude/commands/review-pr.md` :

```
---
allowed-tools: Read, Grep, Glob, Bash(git diff *)
description: Comprehensive code review
---
```

Changed Files

```
!`git diff --name-only HEAD~1`
```

Detailed Changes

```
!`git diff HEAD~1`
```

Review Checklist

Review the above changes for:

1. Code quality and readability
2. Security vulnerabilities
3. Performance implications
4. Test coverage
5. Documentation completeness

Provide specific, actionable feedback organized by priority.

ⓘ Claude Code includes bundled `code-review` and `verify` skills. If you name a custom command after one of them, for example `.claude/commands/code-review.md`, your command shadows the bundled skill and `slash_commands` lists the name once.

Test Runner Command

Create `.claude/commands/test.md` :

```
---
allowed-tools: Bash, Read, Edit
argument-hint: [test-pattern]
description: Run tests with optional pattern
---
```

Run tests matching pattern: \$ARGUMENTS

1. Detect the test framework (Jest, pytest, etc.)
2. Run tests with the provided pattern
3. If tests fail, analyze and fix them
4. Re-run to verify fixes

Use these commands through the SDK:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Run code review
try {
  for await (const message of query({
    prompt: "/review-pr",
    options: { maxTurns: 3 }
  ))) {
    // Process review feedback
  }
} catch (error) {
  // A single-shot query() throws after yielding an error result,
  // so the second query below still runs.
  console.error(`Session ended with an error: ${error}`);
}

// Run specific tests
for await (const message of query({
  prompt: "/test auth",
  options: { maxTurns: 5 }
  ))) {
  // Handle test results
}
```

See Also

- [Slash Commands](#) - Complete slash command documentation
- [Subagents in the SDK](#) - Similar filesystem-based configuration for subagents
- [TypeScript SDK reference](#) - Complete API documentation
- [SDK overview](#) - General SDK concepts
- [CLI reference](#) - Command-line interface